

PlaceUp Career — Automated Job-Application System: Production Architecture

TL;DR

- **Build a hybrid engine:** submit through legitimate ATS APIs where they exist (Greenhouse, Ashby, SmartRecruiters, Workable, Recruitee, Teamtailor, JazzHR, Phenom), and fall back to server-side headless-browser automation only for the ~18 platforms that are web-form-only or require the employer's private key. Keep a mandatory human "review-before-submit" gate on every application.
- **The dedicated per-user inbox pattern** (per-user subaddress on `mail.placeupcareer.com`) via AWS SES inbound → S3 → Lambda/Cloud Function → Firestore) is strongly preferred over Gmail restricted-scope OAuth, which triggers an annual CASA Tier 2 assessment (a bootstrapped SaaS reports paying as little as ~\$540/yr via TAC Security, while third-party assessors are quoted at \$15k-\$75k for larger restricted-scope apps) and a **lifetime 100-user cap that cannot be reset** until the app is verified.
- **This is a legal gray area:** Workday, LinkedIn and most ATS Terms of Service prohibit automated submission. The review-before-submit + per-application human approval model reduces but does not eliminate account/IP-blocking risk. Prefer APIs, throttle hard, never defeat CAPTCHAs — hand off to the user instead.

Key Findings

1. Three intake tiers determine the whole architecture

- **Tier A — legitimate candidate-facing submission API (no employer key needed):** Greenhouse (Job Board API `POST /v1/boards/{token}/jobs/{id}`), Ashby (`applicationForm.submit`), SmartRecruiters (Application API `POST /postings/:uuid/candidates`), Workable, Recruitee (public Careers Site API, **no auth required**), Teamtailor, JazzHR (partner apply API), Phenom (Apply API). These are the lowest-risk, highest-reliability path.
- **Tier B — API exists but requires the employer's OAuth/API key** (unusable for a candidate-side tool): Workday, iCIMS, Oracle Recruiting/Taleo, UKG Pro Recruiting, ADP, Zoho Recruit, Dover, Gem, Pinpoint, SuccessFactors. For our purposes these must be treated as web-form-only.
- **Tier C — web-form-only, headless automation required:** Workday (candidate side), Rippling, Paylocity, Dayforce, Hireology, BambooHR (careers page), Jobvite, BreezyHR, Join, Polymer, plus every Tier B platform.

2. Headless automation: Playwright on Cloud Run Jobs, with a managed-browser fallback

Playwright is the recommended library (best multi-browser support, auto-wait, mature Python + Node SDKs, aligns with the existing Python/FastAPI stack). Run it in a container on **Cloud Run Jobs** for batch application runs and **Cloud Run services** for interactive review/handoff sessions. Budget ≥2 vCPU / 2 GiB per browser and

enlarge `/dev/shm` (Chromium's default 64 MB shm causes crashes). For scale and anti-bot resilience, use a managed browser (Browserbase or Steel.dev, or self-hosted Browserless) rather than defeating detection yourself.

3. The dedicated inbox beats Gmail OAuth decisively

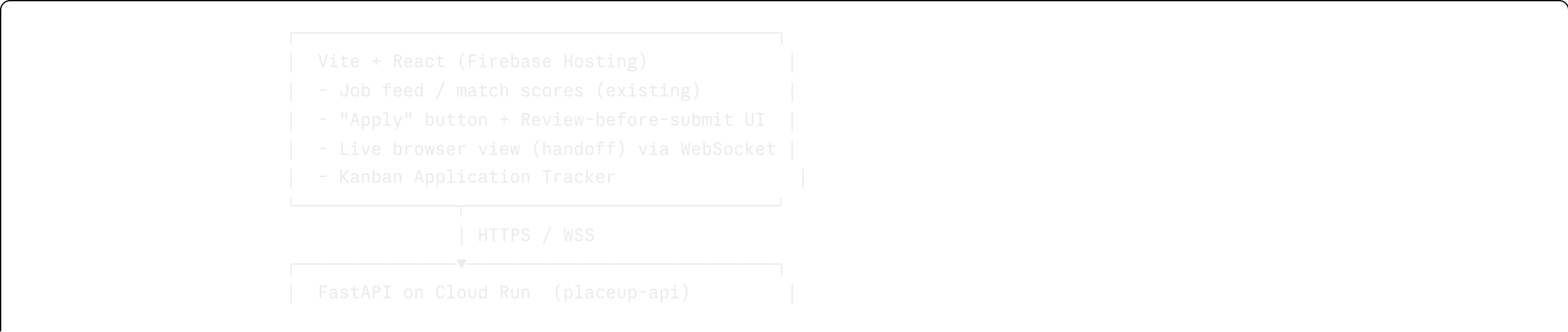
AWS SES inbound (MX → SES receipt rule → S3 → Lambda → parse → forward + store) gives each user `firstname.lastname@mail.placeupcareer.com`. AWS charges \$0.09 per 1,000 incoming 256 KB mail chunks (messages under 256 KB are not billed a chunk), so inbound capture is effectively pennies at any realistic volume. This captures confirmations, status updates and OTP codes with no Google security review.

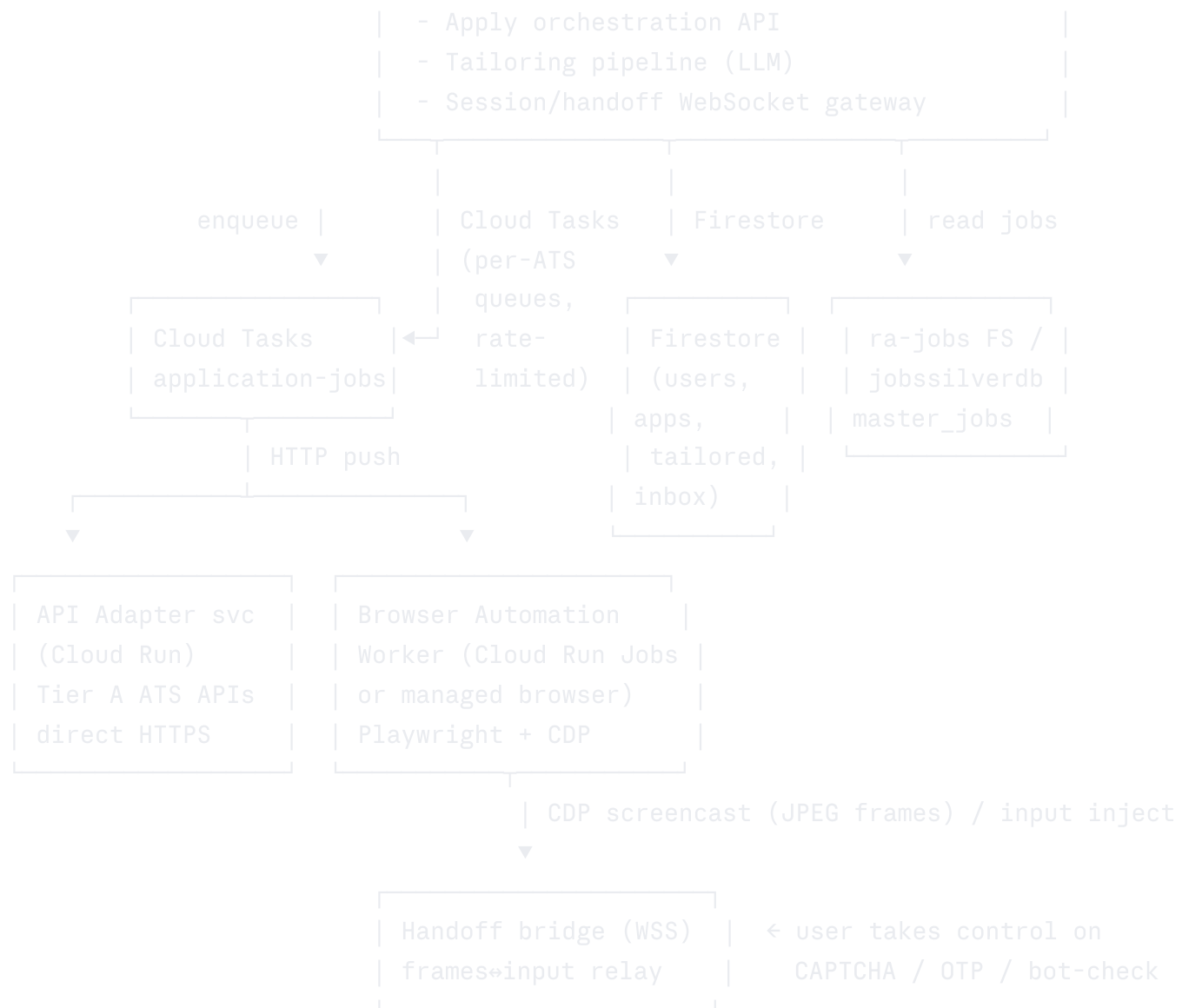
4. This is legally contested territory

Similar tools have a documented history of instability and account risk. Sonara **announced its shutdown on February 1, 2024** (verified via the co-founder's LinkedIn per Teal), was then acquired by BOLD (parent of Zety/LiveCareer/MyPerfectResume) around mid-2024, and relaunched — a cautionary tale about durability. Reported outcome data is mediocre: one Sonara user who submitted 900 applications reported a 5.4% interview rate, and category reviews cite 25–40% of automated applications failing. Workday's End User Agreement explicitly forbids "manual or automated software, devices, or other processes to 'crawl,' 'scrape,' or 'spider'" and using "automated software, scripts, or other methods of accessing or using the Website without Workday's consent." Greenhouse enables CAPTCHA on its forms by default; Workday relies on bot-detection fingerprinting (Imperva/DataDome-style).

Details

A. High-Level Architecture (text diagram)





Email: MX(mail.placeupcareer.com) → AWS SES receipt rule → S3 (raw MIME)
 → Lambda (parse: mailparser) → forward to real email + POST webhook
 → FastAPI → Firestore inbox_messages (+ OTP extraction)

B. Component-by-Component Breakdown

B1. Apply Orchestration API (FastAPI/Cloud Run). Receives "Apply" requests, resolves the job's ATS type from `master_jobs`, decides Tier A (API adapter) vs Tier C (browser worker), creates an `application` record in state `PREPARING`, kicks off the tailoring pipeline, and on user approval enqueues a Cloud Task.

B2. Tailoring pipeline. LLM-based; see section D.

B3. API Adapter service. One adapter module per Tier A ATS. Fetches the job's application schema (e.g., Greenhouse `?questions=true`, SmartRecruiters `GET /postings/:uuid/configuration`, Ashby application form), maps profile → fields, uploads resume/cover letter as multipart or file handle, and submits. Greenhouse's own docs warn that its Job Board submission endpoint does **not** validate required fields server-side ("Validation for required fields must be done on the client side"), so validation must happen in our adapter. SmartRecruiters requires diversity/EEO questions to be presented below all others and privacy-policy consent to be recorded — encode these rules per adapter.

B4. Browser Automation Worker. Playwright container. Reads the ATS adapter config (selectors, step graph), fills the multi-step form, pauses at the review checkpoint, streams frames for approval, submits, captures a confirmation screenshot, writes result to Firestore.

B5. Handoff bridge. WebSocket relay between the headless browser (CDP `Page.startScreencast` → base64 JPEG frames) and the React client; relays user mouse/keyboard events back via `Input.dispatchMouseEvent`/`dispatchKeyEvent` when the user takes control for a CAPTCHA/OTP.

B6. Email ingestion. SES → S3 → Lambda → FastAPI webhook → Firestore (`inbox_messages`).

B7. Application Tracker. Firestore-backed kanban with CSV import/export.

C. ATS Platform Categorization Table (all ~28)

ATS	Intake method	Tier	Integration difficulty
Greenhouse	Public Job Board API — <code>POST /v1/boards/{token}/jobs/{id}</code> , multipart, Base64 Job Board key	A	Low
Lever	Public postings API read-only; apply via <code>postForm</code> web form	C (form)	Medium
Ashby	<code>applicationForm.submit</code> (candidatesWrite) + public posting API	A	Low-Medium
SmartRecruiters	Application API <code>POST /postings/:uuid/candidates</code> (candidate-perspective)	A	Low-Medium
Workable	Public jobs API + candidate-create API	A	Low-Medium

ATS	Intake method	Tier	Integration difficulty
Recruitee	Public Careers Site API, no auth , <code>POST /offers/:slug/candidates</code>	A	Low
Teamtailor	Public API, add candidates (token)	A/B	Medium
JazzHR	Partner apply API (send applications)	A/B	Medium
Phenom	Apply API <code>POST /apply/v2/applications</code> (partner auth) <code>Phenom</code>	A/B	Medium
Workday	No candidate API; web form; bot detection	C	Very High
iCIMS	Apply Framework API (partner/employer creds)	B→C	High
Rippling	<code>push_candidate</code> = onboarding only, blocks customer keys; apply via form	C	High
BambooHR	Careers page; API is employer-key read/write only	C	Medium-High
Jobvite	Employer API; web form for candidates	C	High
BreezyHR	Employer API; web form for candidates	C	Medium
Oracle Recruiting/Taleo	Fusion HCM REST (employer OAuth); else web form	B→C	Very High
Paylocity	Read-only job feed only; web form	C	Medium-High
UltiPro/UKG	Create Application API (UKG-issued creds)	B→C	High
Zoho Recruit	addRecord + associateJobopening (account OAuth)	B→C	High
ADP	Job Applications V2 <code>jobApplication.submit</code> (practitioner scope)	B→C	High
Dover	External API add candidates (employer key) <code>Dover</code>	B→C	High
Gem	ATS API (employer key); apply-write unconfirmed	B→C	High

ATS	Intake method	Tier	Integration difficulty
SAP SuccessFactors	Employer API; web form	C	Very High
Pinpoint	REST API create record (employer key)	B→C	Medium-High
Polymer	Customer API mostly read (employer key) Polymer	B→C	Medium
Dayforce	Career-site web form / Indeed Apply	C	High
Join	Account-token API (write unconfirmed); web form Join	C	Medium
Hireology	Read-only integrations; web form Unified	C	Medium

Reading the table: only **Recruitee** offers a fully open, no-auth, candidate-facing apply API — the ideal case. **Phenom** and **SmartRecruiters** offer candidate-experience apply APIs but need a partner/token relationship. Greenhouse/Ashby/Workable/JazzHR are near-turnkey. Everything else falls back to browser automation, with Workday, Oracle/Taleo and SuccessFactors being the hardest and highest-risk.

D. Resume & Cover-Letter Tailoring Pipeline

1. **Retrieve** the job description from ra-jobs/master_jobs and the user's base resume + structured profile from the Firebase user project.
2. **Extract JD signals** with a cheap model (Gemini 2.5 Flash-Lite at \$0.10/\$0.40 per 1M tokens, or a Groq Llama model for speed): required skills, keywords, seniority, must-have qualifications.
3. **Tailor the resume** with Gemini 2.5 Flash / 3 Flash: re-order and re-phrase bullets to align with JD keywords using **only true facts** from the base resume (mirrors Tsenta's stated no-fabrication approach — "using only true facts from the résumé you uploaded"). Output structured JSON → render to ATS-safe PDF/DOCX (single column, no tables/graphics).
4. **Generate the cover letter** in the same call or a second Flash call.
5. **Score** against the existing job-resume match scorer; show a diff to the user in the review UI.
6. **Cache** the tailored artifacts per (user, company) so re-applications reuse work.

Cost: a tailoring run is roughly 8-15K input + 2-3K output tokens. On Gemini 2.5 Flash (~\$0.30/\$2.50 per 1M) that's about \$0.01-0.02 per job; on Flash-Lite well under \$0.005. The Batch API halves it. Groq Llama 3.3 70B (\$0.59/\$0.79 per 1M) is a latency-optimized alternative. Use a **router**: Flash-Lite for extraction, Flash for generation, Pro only on explicit user request. Keep the router model-agnostic (Gemini 2.0 Flash was deprecated June 1, 2026).

E. Dedicated-Inbox Email Architecture

- **Provider choice:** AWS SES inbound is the recommended primary — mature, cheap (\$0.09 per 1,000 incoming 256 KB chunks; receiving supported only in specific AWS regions, so pin the MX to a supported region such as us-east-1). Cloudflare Email Routing (free inbound forwarding + programmable Email Workers) is a strong low-cost alternative for pure forwarding; Postmark/Mailgun inbound parse-to-webhook is easier to wire but pricier and less flexible.
- **Pattern:** subdomain `mail.placeupcareer.com` MX → SES. A receipt rule with **no recipient condition** acts as a catch-all for every `firstname.lastname@` address, so **no per-user provisioning** is needed. SES stores raw MIME in S3, invokes Lambda, which (a) parses with a mailparser library, (b) forwards a copy to the user's real email, (c) POSTs structured JSON to a FastAPI webhook that writes to `inbox_messages` and runs OTP/verification-code extraction (regex + a small LLM classifier), and (d) links the message to the matching `application` record via sender-domain/subject heuristics.
- **Why not a full mailbox:** we only need capture + forward + parse, not IMAP. Keep it stateless and cheap. (SES has no POP/IMAP; that's a feature here, not a limitation.)

F. Gmail Restricted Scope — the Rejected Alternative

Requesting `https://mail.google.com/` (or other restricted Gmail scopes) forces: (1) OAuth app verification; (2) an **annual** third-party security assessment under Google's CASA framework — **Tier 2 is mandatory for restricted scopes** (Google: applications requesting restricted scopes "must undergo an annual security assessment... awarded a 'Letter of Validation'"); the assessor runs a DAST scan plus a Self-Assessment Questionnaire; and (3) recertification every 12 months.

Costs vary widely by assessor: a bootstrapped SaaS (Orbis) reports paying **\$540** to TAC Security to pass Tier 2, and a developer report cites **\$720** for a TAC "unlimited revalidation" plan — but Unipile (2026) quotes third-party assessors at **\$15,000–\$75,000** for restricted-scope work on larger/more complex apps. Whatever the figure, it recurs **every year**.

Decisively, until the app is verified it is capped at 100 users, and per Google's own console help: *"The user cap applies over the entire lifetime of the project, and it cannot be reset or changed."* For a consumer product targeting thousands of international students, that lifetime cap alone makes restricted-scope Gmail a non-starter — hence the dedicated-inbox pattern.

G. Data Model / Firestore Schemas (illustrative)

```
users/{uid}
  profile: { firstName, lastName, phone, workAuth: "OPT"|"H1B"|..., ... }
  inboxAddress: "first.last@mail.placeupcareer.com"
  realEmail, resumeRef, encryptedFields...
```

```
application_profiles/{uid}    // structured answers reused across ATS
  eeo: {...}, sponsorshipNeeded: bool, addresses:[...], education:[...],
  employment:[...], customAnswers:{questionHash: answer}

tailored_docs/{uid}/{company}
  resumeUrl, coverLetterUrl, jdSignals, matchScore, createdAt

applications/{appId}
  uid, jobId, atsType, tier, status,    // Applied/In-flight/NeedsYou/Failed/
                                       // Skipped/Ghosted/Interviewing/Rejected/Offer
  submissionMethod: "api"|"browser",
  confirmationScreenshotUrl, submittedAt, lastEventAt, history:[...]

ats_adapters/{atsType}        // adapter config
  intakeMethod, selectors, stepGraph, fieldMap, captchaLikely: bool

inbox_messages/{msgId}
  uid, appId?, from, subject, receivedAt, s3Key, parsedText,
  extractedOtp?, classification: "confirmation"|"status"|"otp"|"other"
```

Sensitive fields (PII, any stored credentials) are encrypted at the application layer with Cloud KMS envelope encryption; Firestore security rules restrict access to owner + service account only.

H. Headless Automation Infrastructure & Cost at Three Scales

Recommendation: Playwright in a container. Interactive handoff sessions on a Cloud Run **service** (min-instances tuned so a warm browser is available); batch submissions on Cloud Run **Jobs**. Move to a dedicated GKE pool or a managed-browser vendor once concurrency exceeds what Cloud Run comfortably handles. Do **not** bundle full browser binaries into Cloud Functions — use `playwright-core` + a slim Chromium (`@sparticuz/chromium`) or a managed endpoint. Always `browser.close()` in finally blocks to avoid zombie processes and OOM cascades.

Managed-browser reference pricing (2026): **Browserbase** Starter ~\$20/mo (100 browser hours, 25 concurrent sessions, CAPTCHA-solving included — which we deliberately do **not** use); **Browserless** cloud from ~\$25/mo or self-host from ~€9/mo per server; **Steel.dev** \$29/mo cloud, self-hostable (Apache-2.0, ~0.89s avg session lifecycle in its own benchmark); **Bright Data Scraping Browser** for the hardest anti-bot (400M-IP network, most expensive). A \$6/mo VPS + Playwright is the cheapest DIY but has no residential IPs and gets blocked on Cloudflare/DataDome-protected sites.

- **Dozens of users (≈ 25):** ~5-15 apps/user/week, mostly Tier A. Cloud Run + occasional Cloud Run Jobs. Estimated **\$50-\$150/mo** combined (compute + LLM + email).
- **Hundreds of users (≈ 300):** browser concurrency becomes the constraint. Managed browser (Browserbase/Steel Startup-class tier ~\$99-\$200/mo) or a small GKE pool. Estimated **\$500-\$1,500/mo** all-in (compute + ~\$0.01-0.02/tailor LLM + SES pennies).
- **Thousands of users ($\approx 3,000$):** dedicated GKE browser fleet or an enterprise managed-browser contract; sharded Cloud Tasks queues per ATS; heavy caching of tailored docs. Estimated **\$4,000-\$12,000/mo**, dominated by browser compute and proxy/IP costs — **not** LLM, which stays a few hundred dollars a month with Flash + Batch.

I. Queue & Orchestration

Use **Cloud Tasks** (not Pub/Sub) for application submission: it provides per-queue `max-dispatches-per-second` and `max-concurrent-dispatches` rate limiting, explicit retry caps, scheduling and deduplication — exactly what's needed to throttle per-ATS and avoid tripping volume thresholds. Create **one queue per ATS** so Workday can be paced far more slowly than Greenhouse. Handlers must be idempotent (use the Cloud Tasks task name from the `X-CloudTasks-TaskName` header, or an idempotency key). Reserve Pub/Sub for fan-out events (e.g., "email received" → tracker update + notification). Cloud Scheduler continues to trigger the existing 6-hour scrape.

J. Review-Before-Submit & Graceful Handoff Flow

1. User clicks Apply → tailoring runs → application enters `PREPARING`.
2. System fills the form (API dry-run payload, or browser fills up to — but not including — the final submit button) → state `NEEDS_REVIEW`.
3. React shows the exact payload/screenshot plus the resume/cover-letter diff. The user edits answers, then approves.
4. On approval: Tier A → API submit; Tier C → browser clicks submit.
5. **Graceful handoff:** if the browser worker detects a CAPTCHA, a bot-check challenge, or an OTP/verification prompt (selector + heuristic detection), it sets state `NEEDS_YOU`, freezes automation, and opens a live view — CDP `Page.startScreencast` streams JPEG frames over WebSocket to the user, and the user's clicks/keystrokes are relayed back via CDP input injection. The user solves the challenge themselves (or enters the OTP, which the dedicated inbox may also have captured and surfaced). On completion the user hands control back and automation resumes, or the user finishes the submission manually. **The system never attempts to solve or bypass the security control.** A timeout returns the application to a "paused" state the user can resume later.

K. Phased Implementation Roadmap

- **Phase 0 (weeks 1-3):** ATS adapter framework + Tier A API adapters for Greenhouse, Ashby, SmartRecruiters, Workable, Recruitee. Application data model +

tracker. No browser automation yet.

- **Phase 1 (weeks 4-7):** Tailoring pipeline (Gemini Flash + Groq router) with diff UI and the review-before-submit gate. Dedicated-inbox email (SES) + inbox capture + OTP extraction.
- **Phase 2 (weeks 8-12):** Playwright browser worker on Cloud Run Jobs for Tier C (start with Greenhouse-form and Lever, then Workday). Cloud Tasks per-ATS queues.
- **Phase 3 (weeks 13-16):** Real-time handoff (screenshot + input relay), full kanban statuses (Ghosted/Interviewing/Offer), CSV import/export.
- **Phase 4 (ongoing):** Scale-out (managed browser / GKE), add remaining Tier C adapters by user demand, monitoring & block-rate dashboards, compliance hardening.

L. Security, Privacy & Compliance

- **PII & credentials:** minimize stored credentials — prefer per-application data entry over storing users' ATS logins. Encrypt sensitive fields with Cloud KMS; scope Firestore rules tightly; audit-log every submission with a screenshot/receipt.
- **GDPR/CCPA:** obtain explicit, specific consent for automated submission on the user's behalf; provide data export + deletion; document the email-forwarding processing activity; sign DPAs with SES/Cloudflare. Apply retention limits to `inbox_messages`.
- **Automated-submission risks:** ToS breach → account/IP block; recruiter-side bot detection can silently discard or tag applications as spam; work-authorization/EEO answers must be accurate (the **user**, not the LLM, is responsible for their truthfulness). The human approval gate preserves authenticity and legal defensibility.

M. Recommended Technology Stack (fits existing GCP environment)

- **Backend:** keep FastAPI on Cloud Run (`placeup-api`); add an Apply Orchestration router + WebSocket handoff gateway.
- **Automation:** Playwright (Python, matching the backend) in a container on Cloud Run Jobs; managed-browser (Steel.dev self-hosted or Browserbase) as the scale/anti-bot fallback.
- **Queue:** Cloud Tasks (per-ATS queues) + Cloud Scheduler (existing scrape).
- **Data:** Firestore (`ra-jobs` + user project) for app records/inbox/adapters; Cloud SQL `jobssilverdb`/`master_jobs` for ATS-type resolution; Cloud Storage for tailored docs & confirmation screenshots.
- **LLM:** Gemini 2.5 Flash-Lite / Flash via Vertex AI (region us-east1) with a Groq fallback; Batch API for non-urgent tailoring.
- **Email:** AWS SES inbound + S3 + Lambda (cross-cloud but the mature choice), or Cloudflare Email Routing/Workers if staying single-vendor is preferred.
- **Frontend:** existing Vite + React on Firebase Hosting; add review-before-submit, live-view canvas (renders screenshot frames), and kanban tracker.

Recommendations

1. **Ship Tier A APIs first.** They cover the highest-quality, lowest-risk path (Greenhouse, Ashby, SmartRecruiters, Workable, Recruitee, plus Teamtailor/JazzHR/Phenom as partner integrations). This alone delivers a compelling product with near-zero ban risk.
2. **Treat all Tier B "employer-key" platforms as web-form-only.** Do not attempt to use employer OAuth credentials you don't legitimately hold.
3. **Adopt the dedicated-inbox pattern; formally reject Gmail restricted scope** in the design doc, citing the recurring CASA cost and the lifetime 100-user cap.
4. **Use Cloud Tasks with per-ATS queues and conservative rate limits.** *Threshold to change course:* if the block/failure rate on a given ATS exceeds ~5-10%, slow that queue or disable browser automation for it and route to manual.
5. **Keep the human approval gate non-optional and never build CAPTCHA-solving.** *Threshold:* if a given ATS requires handoff on >50% of attempts, deprioritize automating it.
6. **Cost thresholds:** stay on Cloud Run until interactive browser concurrency regularly exceeds ~25-50 simultaneous sessions; then move to a managed browser or GKE pool.

Caveats

- **ATS API availability changes** — verify each adapter against live docs before launch. Notably, Greenhouse Harvest API v1/v2 are deprecated and unavailable after **August 31, 2026** (migrate to v3). Gem, Polymer and Join write-capability were unconfirmed in public docs and need direct verification.
- **LLM prices are moving targets** (Gemini 2.0 Flash was deprecated June 1, 2026); keep the router model-agnostic.
- **Ban-risk figures are directional.** Reported ~22-23% Workday bot-rejection/restriction rates and the 5.4% interview rate for a 900-application Sonara user come from vendor blogs and reviews with a commercial interest; treat them as indicative, not authoritative.
- **CASA pricing spans two orders of magnitude** (\$540 reported at the low end vs. \$15k-\$75k quoted for larger apps) depending on the assessor and app complexity — but the recurring-annual and lifetime-100-user-cap facts are the decisive ones, and both are non-negotiable.
- **Legality is genuinely unsettled:** automating applications with real data is not itself illegal in the US, but it breaches many platforms' ToS (a civil, not criminal, matter) and can get users blocked. Present this honestly to users and obtain informed consent before submitting on their behalf.